

User Manual

for S32 OCU Driver

Document Number: UM11OCUASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	7
3.1 Requirements	7
3.2 Driver Design Summary	7
3.3 Hardware Resources	8
3.4 Deviations from Requirements	8
3.5 Driver Limitations	9
3.6 Driver usage and configuration tips	9
3.7 Runtime errors	9
3.8 Symbolic Names Disclaimer	9
4 Tresos Configuration Plug-in	10
4.1 Module Ocu	11
4.2 Container OcuConfigurationOfOptionalApis	12
4.3 Parameter OcuDeInitApi	12
4.4 Parameter OcuGetCounterApi	12
4.5 Parameter OcuNotificationSupported	13
4.6 Parameter OcuSetAbsoluteThresholdApi	13
4.7 Parameter OcuSetPinActionApi	14
4.8 Parameter OcuSetPinStateApi	14
4.9 Parameter OcuSetRelativeThresholdApi	15
4.10 Parameter OcuVersionInfoApi	15
4.11 Container OcuConfigSet	15
4.12 Parameter OcuCountdirection	16
4.13 Container OcuChannel	16
4.14 Parameter OcuChannelId	17
4.15 Parameter OcuAssignedHardwareChannel	17
4.16 Parameter HwChannel	18
4.17 Parameter OcuDefaultThreshold	18
4.18 Parameter OcuHardwareTriggeredAdc	19
4.19 Parameter OcuHardwareTriggeredDMA	19
4.20 Parameter OcuMaxCounterValue	20
4.21 Parameter OcuNotification	20

4.22 Parameter OcuOuptutPinUsed	21
4.23 Parameter OcuOutputPinDefaultState	21
4.24 Parameter OcuOutputPinAction	22
4.25 Parameter OcuUseMcuReferencePoint	22
4.26 Parameter OcuChannelTickDuration	23
4.27 Reference OcuChannelEcucPartitionRef	23
4.28 Reference OcuMcuClockReferencePoint	24
4.29 Container OcuHWSpecificSettings	24
4.30 Parameter OcuHardwareChannelId	25
4.31 Parameter OcuFtmModule	25
4.32 Parameter OcuClockSource	26
4.33 Parameter OcuPrescale	26
4.34 Parameter OcuPrescale_Alternate	27
4.35 Parameter OcuDebugMode	27
4.36 Container OcuGeneral	28
4.37 Parameter OcuDevErrorDetect	28
4.38 Parameter OcuEnableDualClockMode	29
4.39 Parameter OcuEnableUserModeSupport	29
4.40 Parameter OcuMulticoreEnabled	30
4.41 Reference OcuEcucPartitionRef	31
4.42 Reference OcuKernelEcucPartitionRef	31
4.43 Container OcuHwResourceConfig	32
4.44 Parameter OcuHwResourceId	32
4.45 Parameter OcuIsrEnable	32
4.46 Parameter OcuChannelIsUsed	33
4.47 Container CommonPublishedInformation	33
4.48 Parameter ArReleaseMajorVersion	34
4.49 Parameter ArReleaseMinorVersion	34
4.50 Parameter ArReleaseRevisionVersion	35
4.51 Parameter ModuleId	35
4.52 Parameter SwMajorVersion	36
4.53 Parameter SwMinorVersion	36
4.54 Parameter SwPatchVersion	36
4.55 Parameter VendorApiInfix	37
4.56 Parameter VendorId	38
5 Module Index	39
5.1 Software Specification	39
6 Module Documentation	40
6.1 Ocu IPL	40

6.1.1 Detailed Description	40
6.1.2 Data Structure Documentation	45
6.1.3 Macro Definition Documentation	46
6.1.4 Types Reference	66
6.1.5 Enum Reference	68
6.1.6 Function Reference	68
6.2 Ocu Driver	70
6.2.1 Detailed Description	70
6.2.2 Data Structure Documentation	72
6.2.3 Macro Definition Documentation	73
6.2.4 Types Reference	80
6.2.5 Enum Reference	81
6.2.6 Function Reference	83



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR OCU driver for S32. AUTOSAR OCU driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR OCU driver requirements and APIs are described in the AUTOSAR OCU driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
AUTOSAR	Automotive Open System Architecture
API	Application Programming Interface
ARTD	Automotive Real Time Drivers
ASR	AUTOSAR
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
EcuM	ECU state Manager
eMIOS	Enhanced Modular IO Subsystem
FTM	FlexTimer
GUI	Graphical User Interface
HLD	High Level Driver
HW	Hardware
IP	Intellectual Property, referred as a hardware design block
IPL	IP Layer
IPW	IP Wrapper Layer
ISR	Interrupt Service Routine
MCU	Micro Controller Unit
N/A	Not Applicable
OCU	Output Compare Unit
OS	Operating System
OSIF	OS Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
RAM	Random Access Memory
ROM	Read-only Memory
SoC	System on Chip
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of OCU Driver	AUTOSAR Release 4.4.0
2	S32G2 Reference Manual	Rev 7, February 2023
3	S32G3 Reference Manual	Rev.3, February 2023
4	S32R45 Reference Manual	Rev. 3, 12/2021

#	Title	Version
5	S32G2 Data Sheet	Rev 6, Dec 2022
6	S32G3 Data Sheet	Rev 2, RC, Feb 2023
7	S32R45 Data Sheet	Rev. 3, RC — 11/2022
8	VR5510 Data Sheet	Rev 5, April 2022
9	S32G2 Errata	Mask Set Errata for Mask 0P77B v3.0
10	S32G2 Errata	Mask Set Errata for Mask 1P77B v1.0
11	S32G3 Errata	Mask Set Errata for Mask 1P72B v2.1
12	S32R45 Errata	Mask Set Errata for Mask P57D, Rev. 2.0

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

For CDD: `_driver>` is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

This driver provides services for initializing and controlling the Output Compare Unit module. OCU module is used to run a counter until a predefined value is reached. When a compare-match occurs, a hardware interrupt and/or a state change of an output pin will be triggered. OCU configuration consists of logic channels exclusively bound to one of the available hardware channels. The Ocu logic channel contains definitions for the initial value of the counter, the maximum counter value (threshold), a notification callback to a user defined function and the behavior of the hardware channel designated pin. A clock source and a prescaler value (optionally a second prescaler) must be set for the hardware channel.

The OCU driver assures reentrancy (single core execution) for the APIs based on the following assumptions:

- the "called-again" API is for a different resource (hardware/logic channel);
- common variables/registers accessed with "rmw" are guarded by Exclusive Areas which need to be correctly implemented in RTE on user side;

3.3 Hardware Resources

#	Hardware IP	Description
1	FTM	FlexTimer

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR OCU Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the OCU Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the OCU driver.

Requirement	Status	Description	Notes
SWS_Ocu_00020	N/S	The detection of production errors cannot be switched off.	DEM errors are not used
SWS_Ocu_00023	N/S	Each variable that shall be accessible by AUTOSAR Debugging, shall be defined as global variable.	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00024	N/S	All type definitions of variables which shall be debugged shall be accessible by the header file Ocu.h .	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00025	N/S	The declaration of variables in the header file shall be such that it is possible to calculate the size of the variables by C-"sizeof".	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00026	N/S	Variables available for debugging shall be described in the respective OCU driver Description.	AUTOSAR Debugging concept is not implemented. In par with PR-MCAL-3330
SWS_Ocu_00125	N/S	If source code for caller and callee of <code>Ocu_GetVersionInfo</code> is available; the OCU driver should realize <code>Ocu_GetVersionInfo</code> as a macro, defined in the module's header file.	<code>Ocu_GetVersionInfo</code> is defined as a function as a common approach in all MCAL drivers.
SWS_Ocu_00156	N/S	These requirements are not applicable to this specification.	Not a requirement

Requirement	Status	Description	Notes
ECUC_Ocu_00168	N/S	Name - OcuKernelEcucPartitionRef - Parent Container - OcuGeneral - Description - Maps the OCU kernel to zero or one ECUC partitions to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the OCU driver is mapped to. Tags: - atp.Status=draft - Multiplicity - 0..1 - Type - Reference to [EcucPartition] - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope: ECU -	Type IV Autosar multicore not implemented for current module (AAI-445), therefore OcuKernelEcucPartitionRef is not supported

3.5 Driver Limitations

None.

3.6 Driver usage and configuration tips

In this chapter, the extra features from our drivers that are not described in the AutoSAR standard are detailed.

3.7 Runtime errors

None.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Ocu](#)
 - Container [OcuConfigurationOfOptionalApis](#)
 - * Parameter [OcuDeInitApi](#)
 - * Parameter [OcuGetCounterApi](#)
 - * Parameter [OcuNotificationSupported](#)
 - * Parameter [OcuSetAbsoluteThresholdApi](#)
 - * Parameter [OcuSetPinActionApi](#)
 - * Parameter [OcuSetPinStateApi](#)
 - * Parameter [OcuSetRelativeThresholdApi](#)
 - * Parameter [OcuVersionInfoApi](#)
 - Container [OcuConfigSet](#)
 - * Parameter [OcuCountdirection](#)
 - * Container [OcuChannel](#)
 - Parameter [OcuChannelId](#)
 - Parameter [OcuAssignedHardwareChannel](#)
 - Parameter [HwChannel](#)
 - Parameter [OcuDefaultThreshold](#)
 - Parameter [OcuHardwareTriggeredAdc](#)
 - Parameter [OcuHardwareTriggeredDMA](#)
 - Parameter [OcuMaxCounterValue](#)
 - Parameter [OcuNotification](#)
 - Parameter [OcuOuptutPinUsed](#)
 - Parameter [OcuOutputPinDefaultState](#)
 - Parameter [OcuOutputPinAction](#)
 - Parameter [OcuUseMcuReferencePoint](#)
 - Parameter [OcuChannelTickDuration](#)
 - Reference [OcuChannelEcucPartitionRef](#)
 - Reference [OcuMcuClockReferencePoint](#)
 - * Container [OcuHWSpecificSettings](#)
 - Parameter [OcuHardwareChannelId](#)

- Parameter [OcuFtmModule](#)
- Parameter [OcuClockSource](#)
- Parameter [OcuPrescale](#)
- Parameter [OcuPrescale_Alternate](#)
- Parameter [OcuDebugMode](#)
- Container [OcuGeneral](#)
 - * Parameter [OcuDevErrorDetect](#)
 - * Parameter [OcuEnableDualClockMode](#)
 - * Parameter [OcuEnableUserModeSupport](#)
 - * Parameter [OcuMulticoreEnabled](#)
 - * Reference [OcuEcucPartitionRef](#)
 - * Reference [OcuKernelEcucPartitionRef](#)
 - * Container [OcuHwResourceConfig](#)
 - Parameter [OcuHwResourceId](#)
 - Parameter [OcuIsrEnable](#)
 - Parameter [OcuChannelIsUsed](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module Ocu

Configuration of Ocu (Output Compare Unit) module.

Included containers:

- `OcuConfigurationOfOptionalApis`
- `OcuConfigSet`
- `OcuGeneral`
- `CommonPublishedInformation`

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container OcuConfigurationOfOptionalApis

Configuration of optional APIs.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter OcuDeInitApi

Adds / removes the service Ocu_DeInit() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.4 Parameter OcuGetCounterApi

Adds / removes the service Ocu_GetCounter() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.5 Parameter OcuNotificationSupported

Adds / removes the services Ocu_EnableNotification() and Ocu_DisableNotification() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.6 Parameter OcuSetAbsoluteThresholdApi

Adds / removes the service Ocu_SetAbsoluteThreshold() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.7 Parameter OcuSetPinActionApi

Adds / removes the service Ocu_SetPinAction() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.8 Parameter OcuSetPinStateApi

Adds / removes the service Ocu_SetPinState() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.9 Parameter OcuSetRelativeThresholdApi

Adds / removes the service Ocu_SetRelativeThreshold() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.10 Parameter OcuVersionInfoApi

Switch to indicate that the Ocu_GetVersionInfo() is supported.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.11 Container OcuConfigSet

This container is the base of a Configuration Set, which contains the configured OCU channels. This way, different configuration sets can be defined for post-build process.

Included subcontainers:

- [OcuChannel](#)
- [OcuHWSpecificSettings](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.12 Parameter OcuCountdirection

This parameter indicates the count direction for the whole OCU driver.

Note This feature does not support down counting. Up-counting is always used as default

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCU_UPCOUNTING
literals	['OCU_DOWNCOUNTING', 'OCU_UPCOUNTING']

4.13 Container OcuChannel

Configuration of an individual OCU channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.14 Parameter OcuChannelId

Channel Id of the OCU channel. This value will be assigned to the symbolic name derived from the OcuChannel container short name. It defines the assignment of the channel to the physical OCU hardware channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.15 Parameter OcuAssignedHardwareChannel

The physical hardware channel that is assigned to this logical channel.

NoteHardware channel should be enumeration parameter and has been replaced by OcuHardwareChannel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	0

4.16 Parameter HwChannel

The Ftm hardware channel that is assigned to this logical channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FTM_0_CH_0
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_1_CH_0', 'FTM_1_CH_1', 'FTM_1_CH_2', 'FTM_1_CH_3', 'FTM_1_CH_4', 'FTM_1_CH_5']

4.17 Parameter OcuDefaultThreshold

Value of comparison threshold used for Initialization.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	10000
max	65535
min	0

4.18 Parameter OcuHardwareTriggeredAdc

This parameter is used to allow the OCU channel to trigger an ADC channel upon compare match, if this is supported by hardware. The value of the parameter represents the ADC physical channel to trigger..

NoteTrigger for ADC should be routed via MCL and has not supported yet in OCU driver. Hardware trigger for ADC could be used by PWM

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.19 Parameter OcuHardwareTriggeredDMA

This parameter is used to allow the OCU channel to trigger a DMA channel upon compare match, if this is supported by hardware. The value of the parameter represents the DMA physical channel to trigger.

NoteHardware trigger for DMA has not supported yet in OCU driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.20 Parameter OcuMaxCounterValue

Maximum value in ticks, the counter of the OCU channel is able to count.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	65535
max	65535
min	1

4.21 Parameter OcuNotification

User callback function

NOTE: please use NULL or NULL_PTR w/o any quotes. If the used string is different from NULL or NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL

4.22 Parameter OcuOuptutPinUsed

Information about the usage of an output pin on this channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.23 Parameter OcuOutputPinDefaultState

The parameter OcuOutputPinDefaultState represents the state that a pin associated with a channel shall be set to after initialisation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCU_LOW
literals	['OCU_HIGH', 'OCU_LOW']

4.24 Parameter OcuOutputPinAction

The parameter OcuOutputPinAction represents the action that a pin associated with a channel shall be set immediately after a compare match event.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCU_DISABLE
literals	['OCU_SET_LOW', 'OCU_SET_HIGH', 'OCU_TOGGLE', 'OCU_DISABLE']

4.25 Parameter OcuUseMcuReferencePoint

This parameter allows for automatic calculation of the Channel Tick duration based on the configured MCU Referenced Clock.

If OcuUseMcuReferencePoint is set to true then selecting a Clock source from MCU is possible and based on that source clock OcuChannelTickDuration can be auto-calculated.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.26 Parameter OcuChannelTickDuration

Specifies the tick duration of the counter of the channel. This parameter is the number of the input clock edges (rising edges or falling edges exclusively) counted each time to increase the counter by one unit.

Note. The default value is calculated using a default prescaler of 1. For a different prescaler value the calculated value should be divided by the said prescaler value.

Note Unit the OCU Channel Tick Duration is nano seconds(ns).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1
max	32768
min	1

4.27 Reference OcuChannelEcucPartitionRef

Maps a OCU channel to zero or multiple ECUC partitions to limit the access to this channel group.

The ECUC partitions referenced are a subset of the ECUC partitions where the OCU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.28 Reference OcuMcuClockReferencePoint

Reference to the Ftm clock source configuration,

which is set in the MCU driver configuration.

Note This parameter is not supported in current implementation.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting← Config/McuClockReferencePoint

4.29 Container OcuHWSpecificSettings

This container contains Ocu-specific parameters for selecting the clock source and setting optional prescalers if supported by hardware.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.30 Parameter OcuHardwareChannelId

Channel ID of the OCU hardware channel. This ID is used to match with OcuChannel/OcuAssignedHardwareChannel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.31 Parameter OcuFtmModule

Selects one Ftm modules available on the platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FTM_0
literals	['FTM_0', 'FTM_1']

4.32 Parameter OcuClockSource

The OCU driver specific clock input for the unit can statically be configured to select different clock sources if provided by hardware. Enumeration literals are defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCU_SYSTEM_CLOCK
literals	['OCU_SYSTEM_CLOCK', 'OCU_EXTERNAL_CLOCK']

4.33 Parameter OcuPrescale

Optional OCU driver specific clock prescale factor, if supported by hardware. Implementation is defined vendor specific.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV4', 'DIV8', 'DIV16', 'DIV32', 'DIV64', 'DIV128']

4.34 Parameter OcuPrescale_Alternate

Select input count source (clock) used for this Ftm module. This parameter will be used by the Ocu_SetClockMode function.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DIV1
literals	['DIV1', 'DIV2', 'DIV4', 'DIV8', 'DIV16', 'DIV32', 'DIV64', 'DIV128']

4.35 Parameter OcuDebugMode

OcuDebugMode

These bits allow select the behavior of the FTM counter, the CH(n)F bit, the channels output, and the writing to the MOD, CNTIN, and C(n)V registers (wave form register)

CNT_ON_OUTPUTS_ON - Continue with normal operation during debug mode.

CNT_STOPED_OUTPUTS_FROZEN - Counter is stopped, channels outputs are frozen, writes to wave form registers bypass the registers buffers

CNT_STOPED_OUTPUTS_SAFE - Counter is stopped, channels outputs are forced to their safe value according to POLn bit, writes to wave form registers bypass the registers buffers

CNT_STOPED_FLAG_SET - Counter is stopped, channels outputs operate with its functionality, writes to wave form registers bypass the registers buffers

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CNT_ON_OUTPUTS_ON
literals	['CNT_ON_OUTPUTS_ON', 'CNT_STOPED_OUTPUTS_FROZEN', 'CNT_STOPED_OUTPUTS_SAFE', 'CNT_STOPED_FLAG_SET']

4.36 Container OcuGeneral

This container contains the module-wide configuration parameters of the OCU Driver.

Included subcontainers:

- [OcuHwResourceConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.37 Parameter OcuDevErrorDetect

Switch for enabling the development error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.38 Parameter OcuEnableDualClockMode

Adds / removes the services Ocu_SetClockMode() from the code. This function is called when the prescaler value needs to be change to maintain same period at different frequency.

Note:

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.39 Parameter OcuEnableUserModeSupport

When this parameter is enabled, the OCU module will adapt to run from User Mode, with the following measures:

Configuring REG_PROT for Ftm IP so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

For more information, please see chapter 5.7 User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.40 Parameter OcuMulticoreEnabled

Switch to enable/disable multi-core feature.

User can choose ENABLE multi-core feature by checking this option, this will force to configure at least 1 ECUC partition in

OcuChannelEcucPartitionRef, and each OCU channel in OcuChannel to configure at least 1 ECUC partition reference

in OcuChannelEcucPartitionRef container to fulfill generating code condition; OR unchecked this option to DISABLE

multi-core feature, performing this action will force user to remove all ECUC partition reference in every OCU channels contained

in OcuChannel and in OcuChannelEcucPartitionRef.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.41 Reference OcuEcucPartitionRef

Maps the OCU driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.42 Reference OcuKernelEcucPartitionRef

Maps the OCU kernel to zero or one ECUC partitions to assign the driver kernel to a certain core.

The ECUC partition referenced is a subset of the ECUC partitions where the OCU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.43 Container OcuHwResourceConfig

List of HW interrupts available for the entire platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	12
upperMultiplicity	12
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.44 Parameter OcuHwResourceId

Id of the HW interrupt service routine available platform wide and usable by the Ocu module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FTM_0_CH_1
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_1_CH_0', 'FTM_1_CH_1', 'FTM_1_CH_2', 'FTM_1_CH_3', 'FTM_1_CH_4', 'FTM_1_CH_5']

4.45 Parameter OcuIsrEnable

Status of the HW Interrupt (true - Interrupt shall be enable platform wide; false - Interrupt shall be disabled platform wide).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.46 Parameter OcuChannelsUsed

This column configures HW channels which are going to be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.47 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.48 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.49 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Property	Value
defaultValue	4
max	4
min	4

4.50 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.51 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	125
max	125
min	125

4.52 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.53 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.54 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.55 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>__>VendorId>__<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.56 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Ocu IPL	40
Ocu Driver	70

Chapter 6

Module Documentation

6.1 Ocu IPL

6.1.1 Detailed Description Ocu FTM driver header file.

Ocu FTM driver header file specific to OCU driver.

Ocu FTM header file support for OCU driver.

FTM driver interface.

FTM specific defines which need to be exported to external application

Data Structures

- struct [Ftm_Ocu_Ip_ChannelConfigType](#)
FTM IP channel specific configuration structure for the Ocu functionality. [More...](#)
- struct [Ftm_Ocu_Ip_SpecificIpConfigType](#)
FTM IP module specific configuration structure for the Ocu functionality. [More...](#)
- struct [Ftm_Ocu_Ip_ModuleConfigType](#)
FTM IP specific configuration structure type. [More...](#)

Macros

- [#define FTM_COMBINE_CH_CTRL_WIDTH](#)
Combine control width for channel combine function.
- [#define OCU_FTM_INVALID_CHANNEL_NUM](#)
invalid FTM channel number
- [#define FTM_0_CH_0](#)
FTM 0 channel 0.
- [#define FTM_0_CH_1](#)
FTM 0 channel 1.
- [#define FTM_0_CH_2](#)

- ```

FTM 0 channel 2.
• #define FTM_0_CH_3
FTM 0 channel 3.
• #define FTM_0_CH_4
FTM 0 channel 4.
• #define FTM_0_CH_5
FTM 0 channel 5.
• #define FTM_0_CH_6
FTM 0 channel 6.
• #define FTM_0_CH_7
FTM 0 channel 7.
• #define FTM_1_CH_0
FTM 1 channel 0.
• #define FTM_1_CH_1
FTM 1 channel 1.
• #define FTM_1_CH_2
FTM 1 channel 2.
• #define FTM_1_CH_3
FTM 1 channel 3.
• #define FTM_1_CH_4
FTM 1 channel 4.
• #define FTM_1_CH_5
FTM 1 channel 5.
• #define FTM_1_CH_6
FTM 1 channel 6.
• #define FTM_1_CH_7
FTM 1 channel 7.
• #define FTM_2_CH_0
FTM 2 channel 0.
• #define FTM_2_CH_1
FTM 2 channel 1.
• #define FTM_2_CH_2
FTM 2 channel 2.
• #define FTM_2_CH_3
FTM 2 channel 3.
• #define FTM_2_CH_4
FTM 2 channel 4.
• #define FTM_2_CH_5
FTM 2 channel 5.
• #define FTM_2_CH_6
FTM 2 channel 6.
• #define FTM_2_CH_7
FTM 2 channel 7.
• #define FTM_3_CH_0
FTM 3 channel 0.
• #define FTM_3_CH_1
FTM 3 channel 1.

```

- `#define FTM_3_CH_2`  
*FTM 3 channel 2.*
- `#define FTM_3_CH_3`  
*FTM 3 channel 3.*
- `#define FTM_3_CH_4`  
*FTM 3 channel 4.*
- `#define FTM_3_CH_5`  
*FTM 3 channel 5.*
- `#define FTM_3_CH_6`  
*FTM 3 channel 6.*
- `#define FTM_3_CH_7`  
*FTM 3 channel 7.*
- `#define FTM_4_CH_0`  
*FTM 4 channel 0.*
- `#define FTM_4_CH_1`  
*FTM 4 channel 1.*
- `#define FTM_4_CH_2`  
*FTM 4 channel 2.*
- `#define FTM_4_CH_3`  
*FTM 4 channel 3.*
- `#define FTM_4_CH_4`  
*FTM 4 channel 4.*
- `#define FTM_4_CH_5`  
*FTM 4 channel 5.*
- `#define FTM_4_CH_6`  
*FTM 4 channel 6.*
- `#define FTM_4_CH_7`  
*FTM 4 channel 7.*
- `#define FTM_5_CH_0`  
*FTM 5 channel 0.*
- `#define FTM_5_CH_1`  
*FTM 5 channel 1.*
- `#define FTM_5_CH_2`  
*FTM 5 channel 2.*
- `#define FTM_5_CH_3`  
*FTM 5 channel 3.*
- `#define FTM_5_CH_4`  
*FTM 5 channel 4.*
- `#define FTM_5_CH_5`  
*FTM 5 channel 5.*
- `#define FTM_5_CH_6`  
*FTM 5 channel 6.*
- `#define FTM_5_CH_7`  
*FTM 5 channel 7.*
- `#define FTM_6_CH_0`  
*FTM 6 channel 0.*
- `#define FTM_6_CH_1`

- *FTM 6 channel 1.*
- `#define FTM_6_CH_2`  
*FTM 6 channel 2.*
- `#define FTM_6_CH_3`  
*FTM 6 channel 3.*
- `#define FTM_6_CH_4`  
*FTM 6 channel 4.*
- `#define FTM_6_CH_5`  
*FTM 6 channel 5.*
- `#define FTM_6_CH_6`  
*FTM 6 channel 6.*
- `#define FTM_6_CH_7`  
*FTM 6 channel 7.*
- `#define FTM_7_CH_0`  
*FTM 7 channel 0.*
- `#define FTM_7_CH_1`  
*FTM 7 channel 1.*
- `#define FTM_7_CH_2`  
*FTM 7 channel 2.*
- `#define FTM_7_CH_3`  
*FTM 7 channel 3.*
- `#define FTM_7_CH_4`  
*FTM 7 channel 4.*
- `#define FTM_7_CH_5`  
*FTM 7 channel 5.*
- `#define FTM_7_CH_6`  
*FTM 7 channel 6.*
- `#define FTM_7_CH_7`  
*FTM 7 channel 7.*
- `#define FTM_BDM_MODE_00`  
*Options for the FlexTimer behavior in BDM Mode.*
- `#define FTM_BDM_MODE_01`
- `#define FTM_BDM_MODE_10`
- `#define FTM_BDM_MODE_11`
- `#define FTM_CLOCK_SOURCE_NONE`  
*FlexTimer Clock Source Selection.*
- `#define FTM_CLOCK_SOURCE_SYSTEMCLK`
- `#define FTM_CLOCK_SOURCE_FIXEDCLK`
- `#define FTM_CLOCK_SOURCE_EXTERNALCLK`
- `#define FTM_CLOCK_DIVID_BY_1`  
*FlexTimer Prescale Factor Selections bits (PS).*
- `#define FTM_CLOCK_DIVID_BY_2`
- `#define FTM_CLOCK_DIVID_BY_4`
- `#define FTM_CLOCK_DIVID_BY_8`
- `#define FTM_CLOCK_DIVID_BY_16`
- `#define FTM_CLOCK_DIVID_BY_32`
- `#define FTM_CLOCK_DIVID_BY_64`
- `#define FTM_CLOCK_DIVID_BY_128`

- `#define FTM_OCU_SET_LOW`  
*Edge Pin Action values.*
- `#define FTM_OCU_SET_HIGH`  
*The channel pin will be set LOW upon compare match.*
- `#define FTM_OCU_SET_TOGGLE`  
*The channel pin will be set to the opposite of its current level HIGH upon compare match.*
- `#define FTM_OCU_SET_DISABLE`  
*The channel pin will remain at its current level upon compare match.*
- `#define OCU_FTM_LOW`  
*Pin State levels.*
- `#define OCU_FTM_HIGH`  
*Ocu Pin level is logic high.*
- `#define FTM_OCU_PRIMARY_PRESCALER`  
*Prescaler values.*
- `#define FTM_OCU_ALTERNATIVE_PRESCALER`  
*Selected value is the alternative configured prescaler.*
- `#define OCU_FTM_CM_IN_REF_INTERVAL`  
*Ocu Return values.*
- `#define OCU_FTM_CM_OUT_REF_INTERVAL`  
*The compare match will not occur inside the current Reference Interval.*

## Types Reference

- `typedef uint8 Ftm_Ocu_Ip_PinActionType`  
*Edge Pin Action type.*
- `typedef uint8 Ftm_Ocu_Ip_PinStateType`  
*Pin State level.*
- `typedef uint8 Ftm_Ocu_Ip_SelectPrescalerType`  
*Prescaler type.*
- `typedef uint8 Ftm_Ocu_Ip_ReturnType`  
*Ocu Return Type.*
- `typedef FTM_Type Ftm_Ocu_Ip_xRegLayoutType`
- `typedef void(* Ftm_Ocu_Ip_CallbackType) (uint16 CallbackParam)`  
*Channel notification typedef.*

## Enum Reference

- `enum Ftm_Ocu_Ip_OutputCompareModeType`  
*FlexTimer output compare edge mode. Toggle, clear or set.*

## Function Reference

- void [Ftm\\_Ocu\\_Ip\\_Init](#) (const [Ftm\\_Ocu\\_Ip\\_ModuleConfigType](#) \*const pFtmIpConfig)  
*This function initializes all internals variables of the driver.*
- void [Ftm\\_Ocu\\_Ip\\_StartChannel](#) (uint8 InstNum, uint8 ChNum)  
*Ocu driver function for starting the FTM timer channel.*
- void [Ftm\\_Ocu\\_Ip\\_StopChannel](#) (uint8 InstNum, uint8 ChNum)  
*Ocu driver function for stopping the FTM timer channel.*
- #define [FTM\\_MOD\\_MASK\\_U8](#)  
*FTM channels defines.*
- #define [OCU\\_FTM\\_OUTPUTDISABLED](#)  
*FTM register define used to configure counter clock source.*

## 6.1.2 Data Structure Documentation

### 6.1.2.1 struct Ftm\_Ocu\_Ip\_ChannelConfigType

FTM IP channel specific configuration structure for the Ocu functionality.

Definition at line 606 of file Ftm\_Ocu\_Ip\_Types.h.

Data Fields

| Type         | Name                | Description                                                                                                                                                                                                                                                                                             |
|--------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| const uint16 | HwChannel           | Assigned FTM channel Id.                                                                                                                                                                                                                                                                                |
| const uint16 | DefaultThreshold    | Compare match threshold for the current channel.                                                                                                                                                                                                                                                        |
| const uint8  | ChannelControlValue | FTM channel parameters. Bits 7 --> 1: Output Pin is Used / 0: Output Pin is not used Bit 6 --> Default Pin State: 0: PIN_LOW; 1: PIN_HIGH Bits 5 .. 4 --> Pin Action behaviour on compare match: SET_PIN_LOW; 2: SET_PIN_HIGH; 1: PIN_TOGGLE; 0: OUTPUT_DISABLE Bits 3 .. 0 --> Reserved for future use |

### 6.1.2.2 struct Ftm\_Ocu\_Ip\_SpecificIpConfigType

FTM IP module specific configuration structure for the Ocu functionality.

Definition at line 627 of file Ftm\_Ocu\_Ip\_Types.h.



### Data Fields

| Type         | Name               | Description                                                                                                                                             |
|--------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| const uint8  | ModuleId           | Assigned FTM module Id.                                                                                                                                 |
| const uint16 | MaxCounterValue    | Assigned FTM module Id.                                                                                                                                 |
| const uint8  | ModuleControlValue | FTM module parameters. Bits 7 .. 6 --> Clock source Bits 5 .. 3 --> Normal prescale Bit 2 .. 1 --> Debug mode config Bits 0 --> Reserved for future use |

### 6.1.2.3 struct Ftm\_Ocu\_Ip\_ModuleConfigType

FTM IP specific configuration structure type.

Definition at line 648 of file Ftm\_Ocu\_Ip\_Types.h.

### Data Fields

| Type                                   | Name                | Description                                      |
|----------------------------------------|---------------------|--------------------------------------------------|
| const uint8                            | NumChannels         | Number of FTM channels in the Ocu configuration. |
| const uint8                            | NumModules          | Number of FTM modules in the Ocu configuration.  |
| const Ftm_Ocu_Ip_ChannelConfigType*    | pcxChannelsConfig[] | Pointer to the configured channels for FTM.      |
| const Ftm_Ocu_Ip_SpecificIpConfigType* | pcxModulesConfig[]  | Pointer to the configured channels for FTM.      |

## 6.1.3 Macro Definition Documentation

### 6.1.3.1 FTM\_COMBINE\_CH\_CTRL\_WIDTH

```
#define FTM_COMBINE_CH_CTRL_WIDTH
```

Combine control width for channel combine function.

Definition at line 88 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.2 OCU\_FTM\_INVALID\_CHANNEL\_NUM

```
#define OCU_FTM_INVALID_CHANNEL_NUM
```

invalid FTM channel number

Definition at line 91 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.3 FTM\_MOD\_MASK\_U8

```
#define FTM_MOD_MASK_U8
```

FTM channels defines.

There are defines used for the FTM channel encoding -> channel\_id

Note

a FTM module generally has only 6 channels so by using this method there will be gaps of 2 channels between different FTM modules because [ 5 bit | 3 bit ] [ module id: 31 ... 0 | channel id: 7 ... 0 ]

Definition at line 103 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.4 FTM\_0\_CH\_0

```
#define FTM_0_CH_0
```

FTM 0 channel 0.

Definition at line 116 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.5 FTM\_0\_CH\_1

```
#define FTM_0_CH_1
```

FTM 0 channel 1.

Definition at line 120 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.6 FTM\_0\_CH\_2

```
#define FTM_0_CH_2
```

FTM 0 channel 2.

Definition at line 124 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.7 FTM\_0\_CH\_3

```
#define FTM_0_CH_3
```

FTM 0 channel 3.

Definition at line 128 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.8 FTM\_0\_CH\_4

```
#define FTM_0_CH_4
```

FTM 0 channel 4.

Definition at line 132 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.9 FTM\_0\_CH\_5

```
#define FTM_0_CH_5
```

FTM 0 channel 5.

Definition at line 136 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.10 FTM\_0\_CH\_6

```
#define FTM_0_CH_6
```

FTM 0 channel 6.

Definition at line 140 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.11 FTM\_0\_CH\_7

```
#define FTM_0_CH_7
```

FTM 0 channel 7.

Definition at line 144 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.12 FTM\_1\_CH\_0

```
#define FTM_1_CH_0
```

FTM 1 channel 0.

Definition at line 149 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.13 FTM\_1\_CH\_1

```
#define FTM_1_CH_1
```

FTM 1 channel 1.

Definition at line 153 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.14 FTM\_1\_CH\_2

```
#define FTM_1_CH_2
```

FTM 1 channel 2.

Definition at line 157 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.15 FTM\_1\_CH\_3

```
#define FTM_1_CH_3
```

FTM 1 channel 3.

Definition at line 161 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.16 FTM\_1\_CH\_4

```
#define FTM_1_CH_4
```

FTM 1 channel 4.

Definition at line 165 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.17 FTM\_1\_CH\_5

```
#define FTM_1_CH_5
```

FTM 1 channel 5.

Definition at line 169 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.18 FTM\_1\_CH\_6

```
#define FTM_1_CH_6
```

FTM 1 channel 6.

Definition at line 173 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.19 FTM\_1\_CH\_7

```
#define FTM_1_CH_7
```

FTM 1 channel 7.

Definition at line 177 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.20 FTM\_2\_CH\_0

```
#define FTM_2_CH_0
```

FTM 2 channel 0.

Definition at line 182 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.21 FTM\_2\_CH\_1

```
#define FTM_2_CH_1
```

FTM 2 channel 1.

Definition at line 186 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.22 FTM\_2\_CH\_2

```
#define FTM_2_CH_2
```

FTM 2 channel 2.

Definition at line 190 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.23 FTM\_2\_CH\_3

```
#define FTM_2_CH_3
```

FTM 2 channel 3.

Definition at line 194 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.24 FTM\_2\_CH\_4

```
#define FTM_2_CH_4
```

FTM 2 channel 4.

Definition at line 198 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.25 FTM\_2\_CH\_5

```
#define FTM_2_CH_5
```

FTM 2 channel 5.

Definition at line 202 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.26 FTM\_2\_CH\_6

```
#define FTM_2_CH_6
```

FTM 2 channel 6.

Definition at line 206 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.27 FTM\_2\_CH\_7

```
#define FTM_2_CH_7
```

FTM 2 channel 7.

Definition at line 210 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.28 FTM\_3\_CH\_0

```
#define FTM_3_CH_0
```

FTM 3 channel 0.

Definition at line 215 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.29 FTM\_3\_CH\_1

```
#define FTM_3_CH_1
```

FTM 3 channel 1.

Definition at line 219 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.30 FTM\_3\_CH\_2

```
#define FTM_3_CH_2
```

FTM 3 channel 2.

Definition at line 223 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.31 FTM\_3\_CH\_3

```
#define FTM_3_CH_3
```

FTM 3 channel 3.

Definition at line 227 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.32 FTM\_3\_CH\_4

```
#define FTM_3_CH_4
```

FTM 3 channel 4.

Definition at line 231 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.33 FTM\_3\_CH\_5

```
#define FTM_3_CH_5
```

FTM 3 channel 5.

Definition at line 235 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.34 FTM\_3\_CH\_6

```
#define FTM_3_CH_6
```

FTM 3 channel 6.

Definition at line 239 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.35 FTM\_3\_CH\_7

```
#define FTM_3_CH_7
```

FTM 3 channel 7.

Definition at line 243 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.36 FTM\_4\_CH\_0

```
#define FTM_4_CH_0
```

FTM 4 channel 0.

Definition at line 248 of file Ftm\_Ocu\_Ip\_Types.h.



### 6.1.3.37 FTM\_4\_CH\_1

```
#define FTM_4_CH_1
```

FTM 4 channel 1.

Definition at line 252 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.38 FTM\_4\_CH\_2

```
#define FTM_4_CH_2
```

FTM 4 channel 2.

Definition at line 256 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.39 FTM\_4\_CH\_3

```
#define FTM_4_CH_3
```

FTM 4 channel 3.

Definition at line 260 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.40 FTM\_4\_CH\_4

```
#define FTM_4_CH_4
```

FTM 4 channel 4.

Definition at line 264 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.41 FTM\_4\_CH\_5

```
#define FTM_4_CH_5
```

FTM 4 channel 5.

Definition at line 268 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.42 FTM\_4\_CH\_6

```
#define FTM_4_CH_6
```

FTM 4 channel 6.

Definition at line 272 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.43 FTM\_4\_CH\_7

```
#define FTM_4_CH_7
```

FTM 4 channel 7.

Definition at line 276 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.44 FTM\_5\_CH\_0

```
#define FTM_5_CH_0
```

FTM 5 channel 0.

Definition at line 281 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.45 FTM\_5\_CH\_1

```
#define FTM_5_CH_1
```

FTM 5 channel 1.

Definition at line 285 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.46 FTM\_5\_CH\_2

```
#define FTM_5_CH_2
```

FTM 5 channel 2.

Definition at line 289 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.47 FTM\_5\_CH\_3

```
#define FTM_5_CH_3
```

FTM 5 channel 3.

Definition at line 293 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.48 FTM\_5\_CH\_4

```
#define FTM_5_CH_4
```

FTM 5 channel 4.

Definition at line 297 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.49 FTM\_5\_CH\_5

```
#define FTM_5_CH_5
```

FTM 5 channel 5.

Definition at line 301 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.50 FTM\_5\_CH\_6

```
#define FTM_5_CH_6
```

FTM 5 channel 6.

Definition at line 305 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.51 FTM\_5\_CH\_7

```
#define FTM_5_CH_7
```

FTM 5 channel 7.

Definition at line 309 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.52 FTM\_6\_CH\_0

```
#define FTM_6_CH_0
```

FTM 6 channel 0.

Definition at line 314 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.53 FTM\_6\_CH\_1

```
#define FTM_6_CH_1
```

FTM 6 channel 1.

Definition at line 318 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.54 FTM\_6\_CH\_2

```
#define FTM_6_CH_2
```

FTM 6 channel 2.

Definition at line 322 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.55 FTM\_6\_CH\_3

```
#define FTM_6_CH_3
```

FTM 6 channel 3.

Definition at line 326 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.56 FTM\_6\_CH\_4

```
#define FTM_6_CH_4
```

FTM 6 channel 4.

Definition at line 330 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.57 FTM\_6\_CH\_5

```
#define FTM_6_CH_5
```

FTM 6 channel 5.

Definition at line 334 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.58 FTM\_6\_CH\_6

```
#define FTM_6_CH_6
```

FTM 6 channel 6.

Definition at line 338 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.59 FTM\_6\_CH\_7

```
#define FTM_6_CH_7
```

FTM 6 channel 7.

Definition at line 342 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.60 FTM\_7\_CH\_0

```
#define FTM_7_CH_0
```

FTM 7 channel 0.

Definition at line 347 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.61 FTM\_7\_CH\_1

```
#define FTM_7_CH_1
```

FTM 7 channel 1.

Definition at line 351 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.62 FTM\_7\_CH\_2

```
#define FTM_7_CH_2
```

FTM 7 channel 2.

Definition at line 355 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.63 FTM\_7\_CH\_3

```
#define FTM_7_CH_3
```

FTM 7 channel 3.

Definition at line 359 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.64 FTM\_7\_CH\_4

```
#define FTM_7_CH_4
```

FTM 7 channel 4.

Definition at line 363 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.65 FTM\_7\_CH\_5

```
#define FTM_7_CH_5
```

FTM 7 channel 5.

Definition at line 367 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.66 FTM\_7\_CH\_6

```
#define FTM_7_CH_6
```

FTM 7 channel 6.

Definition at line 371 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.67 FTM\_7\_CH\_7

```
#define FTM_7_CH_7
```

FTM 7 channel 7.

Definition at line 375 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.68 OCU\_FTM\_OUTPUTDISABLED

```
#define OCU_FTM_OUTPUTDISABLED
```

FTM register define used to configure counter clock source.

Definition at line 382 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.69 FTM\_BDM\_MODE\_00

```
#define FTM_BDM_MODE_00
```

Options for the FlexTimer behavior in BDM Mode.

FTM counter stopped, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers

Definition at line 442 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.70 FTM\_BDM\_MODE\_01

```
#define FTM_BDM_MODE_01
```

FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are forced to their safe value, writes to MOD, CNTIN and C(n)V registers bypass the register buffers

Definition at line 445 of file Ftm\_Ocu\_Ip\_Types.h.

**6.1.3.71 FTM\_BDM\_MODE\_10**

```
#define FTM_BDM_MODE_10
```

FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are frozen when chip enters in BDM mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers

Definition at line 448 of file Ftm\_Ocu\_Ip\_Types.h.

**6.1.3.72 FTM\_BDM\_MODE\_11**

```
#define FTM_BDM_MODE_11
```

FTM counter in functional mode, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD, CNTIN and C(n)V registers is in fully functional mode

Definition at line 451 of file Ftm\_Ocu\_Ip\_Types.h.

**6.1.3.73 FTM\_CLOCK\_SOURCE\_NONE**

```
#define FTM_CLOCK_SOURCE_NONE
```

FlexTimer Clock Source Selection.

None use clock for FTM

Definition at line 457 of file Ftm\_Ocu\_Ip\_Types.h.

**6.1.3.74 FTM\_CLOCK\_SOURCE\_SYSTEMCLK**

```
#define FTM_CLOCK_SOURCE_SYSTEMCLK
```

System clock

Definition at line 458 of file Ftm\_Ocu\_Ip\_Types.h.



### 6.1.3.75 FTM\_CLOCK\_SOURCE\_FIXEDCLK

```
#define FTM_CLOCK_SOURCE_FIXEDCLK
```

Fixed clock

Definition at line 459 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.76 FTM\_CLOCK\_SOURCE\_EXTERNALCLK

```
#define FTM_CLOCK_SOURCE_EXTERNALCLK
```

External clock

Definition at line 460 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.77 FTM\_CLOCK\_DIVID\_BY\_1

```
#define FTM_CLOCK_DIVID_BY_1
```

FlexTimer Prescale Factor Selections bits (PS).

Selects one of 8 division factors for the clock source selected by CLKS. The new prescaler factor affects the clock source on the next FTM input clock cycle after the new value is updated into the register bits. This field is write protected. It can be written only when MODE[WPDIS] = 1. 000b - Divide by 1 001b - Divide by 2 010b - Divide by 4 011b - Divide by 8 100b - Divide by 16 101b - Divide by 32 110b - Divide by 64 111b - Divide by 128 Divide by 1

Definition at line 478 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.78 FTM\_CLOCK\_DIVID\_BY\_2

```
#define FTM_CLOCK_DIVID_BY_2
```

Divide by 2

Definition at line 479 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.79 FTM\_CLOCK\_DIVID\_BY\_4

```
#define FTM_CLOCK_DIVID_BY_4
```

Divide by 4

Definition at line 480 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.80 FTM\_CLOCK\_DIVID\_BY\_8

```
#define FTM_CLOCK_DIVID_BY_8
```

Divide by 8

Definition at line 481 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.81 FTM\_CLOCK\_DIVID\_BY\_16

```
#define FTM_CLOCK_DIVID_BY_16
```

Divide by 16

Definition at line 482 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.3.82 FTM\_CLOCK\_DIVID\_BY\_32

```
#define FTM_CLOCK_DIVID_BY_32
```

Divide by 32

Definition at line 483 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.83 FTM\_CLOCK\_DIVID\_BY\_64

```
#define FTM_CLOCK_DIVID_BY_64
```

Divide by 64

Definition at line 484 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.84 FTM\_CLOCK\_DIVID\_BY\_128

```
#define FTM_CLOCK_DIVID_BY_128
```

Divide by 128

Definition at line 485 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.85 FTM\_OCU\_SET\_LOW

```
#define FTM_OCU_SET_LOW
```

Edge Pin Action values.

Automatic action (by hardware) to be performed on a pin attached to an OCU channel.

The channel pin will be set HIGH upon compare match.

Definition at line 492 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.86 FTM\_OCU\_SET\_HIGH

```
#define FTM_OCU_SET_HIGH
```

The channel pin will be set LOW upon compare match.

Definition at line 494 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.87 FTM\_OCU\_SET\_TOGGLE

```
#define FTM_OCU_SET_TOGGLE
```

The channel pin will be set to the opposite of its current level HIGH upon compare match.

Definition at line 496 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.88 FTM\_OCU\_SET\_DISABLE

```
#define FTM_OCU_SET_DISABLE
```

The channel pin will remain at its current level upon compare match.

Definition at line 498 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.89 OCU\_FTM\_LOW

```
#define OCU_FTM_LOW
```

Pin State levels.

Output state of the pin linked to an OCU channel.

Ocu Pin level is logic low

Definition at line 506 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.90 OCU\_FTM\_HIGH

```
#define OCU_FTM_HIGH
```

Ocu Pin level is logic high.

Definition at line 508 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.91 FTM\_OCU\_PRIMARY\_PRESCALER

```
#define FTM_OCU_PRIMARY_PRESCALER
```

Prescaler values.

Possible values of prescallers used to configure base-clock timers

Selected value is the default/primary prescaler

Definition at line 516 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.92 FTM\_OCU\_ALTERNATIVE\_PRESCALER

```
#define FTM_OCU_ALTERNATIVE_PRESCALER
```

Selected value is the alternative configured prescaler.

Definition at line 518 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.93 OCU\_FTM\_CM\_IN\_REF\_INTERVAL

```
#define OCU_FTM_CM_IN_REF_INTERVAL
```

Ocu Return values.

Return information after setting a new threshold value.

The compare match will occur inside the current Reference Interval.

Definition at line 526 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.3.94 OCU\_FTM\_CM\_OUT\_REF\_INTERVAL

```
#define OCU_FTM_CM_OUT_REF_INTERVAL
```

The compare match will not occur inside the current Reference Interval.

Definition at line 528 of file Ftm\_Ocu\_Ip\_Types.h.

## 6.1.4 Types Reference

#### 6.1.4.1 Ftm\_Ocu\_Ip\_PinActionType

```
typedef uint8 Ftm_Ocu_Ip_PinActionType
```

Edge Pin Action type.

Automatic action (by hardware) to be performed on a pin attached to an OCU channel.

Definition at line 551 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.4.2 Ftm\_Ocu\_Ip\_PinStateType

```
typedef uint8 Ftm_Ocu_Ip_PinStateType
```

Pin State level.

Output state of the pin linked to an OCU channel.

Definition at line 558 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.4.3 Ftm\_Ocu\_Ip\_SelectPrescalerType

```
typedef uint8 Ftm_Ocu_Ip_SelectPrescalerType
```

Prescaler type.

Specifies the possible values of prescallers used to configure base-clock timers

Definition at line 565 of file Ftm\_Ocu\_Ip\_Types.h.

#### 6.1.4.4 Ftm\_Ocu\_Ip\_ReturnType

```
typedef uint8 Ftm_Ocu_Ip_ReturnType
```

Ocu Return Type.

Return information after setting a new threshold value.

Definition at line 572 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.4.5 Ftm\_Ocu\_Ip\_xRegLayoutType

```
typedef FTM_Type Ftm_Ocu_Ip_xRegLayoutType
```

Redefine FTM Register Layout Typedef from header file to comply with coding guidelines

Definition at line 575 of file Ftm\_Ocu\_Ip\_Types.h.

### 6.1.4.6 Ftm\_Ocu\_Ip\_CallbackType

```
typedef void(* Ftm_Ocu_Ip_CallbackType) (uint16 CallbackParam)
```

Channel notification typedef.

Definition at line 578 of file Ftm\_Ocu\_Ip\_Types.h.

## 6.1.5 Enum Reference

### 6.1.5.1 Ftm\_Ocu\_Ip\_OutputCompareModeType

```
enum Ftm_Ocu_Ip_OutputCompareModeType
```

FlexTimer output compare edge mode. Toggle, clear or set.

Definition at line 535 of file Ftm\_Ocu\_Ip\_Types.h.

## 6.1.6 Function Reference

### 6.1.6.1 Ftm\_Ocu\_Ip\_Init()

```
void Ftm_Ocu_Ip_Init (
 const Ftm_Ocu_Ip_ModuleConfigType *const pFtmIpConfig)
```

This function initializes all internal variables of the driver.

This function will initialize with default values all FTM registers. Will loop through all FTM modules and channels in the configuration structure and will setup required registers for each channel and module

Parameters

|    |                     |                                             |
|----|---------------------|---------------------------------------------|
| in | <i>pFtmIpConfig</i> | - Pointer to FTM IP configuration structure |
|----|---------------------|---------------------------------------------|

### 6.1.6.2 Ftm\_Ocu\_Ip\_StartChannel()

```
void Ftm_Ocu_Ip_StartChannel (
 uint8 InstNum,
 uint8 ChNum)
```

Ocu driver function for starting the FTM timer channel.

This function: Configure ELSB:ELSA Release software control Clear flag and enable interrupt

Parameters

|    |                |                                      |
|----|----------------|--------------------------------------|
| in | <i>InstNum</i> | - FTM instance number                |
| in | <i>ChNum</i>   | - Channel number in the FTM instance |

### 6.1.6.3 Ftm\_Ocu\_Ip\_StopChannel()

```
void Ftm_Ocu_Ip_StopChannel (
 uint8 InstNum,
 uint8 ChNum)
```

Ocu driver function for stopping the FTM timer channel.

This function:

- stop the timer channel by writing the ELSB:ELSA = 0:0
- Clear flag and disable interrupt

Parameters

|    |                |                                      |
|----|----------------|--------------------------------------|
| in | <i>InstNum</i> | - FTM instance number                |
| in | <i>ChNum</i>   | - Channel number in the FTM instance |



## 6.2 Ocu Driver

### 6.2.1 Detailed Description

Ocu HLD module API header.

Ocu HLD generic types file.

Ocu HLD notifications API header.

Ocu HLD Environment configuration header file.

Ocu module API header, containing Autosar API and function that are exported by OCU driver

Ocu environment configuration header.

Ocu notifications API header.

OCU header file containing generic Ocu types and structures.

## Data Structures

- struct [Ocu\\_ChannelConfigType](#)  
*Ocu high level channel configuration structure. [More...](#)*
- struct [Ocu\\_ConfigType](#)  
*Ocu high level configuration structure. [More...](#)*

## Macros

- `#define` [OCU\\_STOP\\_SEC\\_CONFIG\\_DATA\\_UNSPECIFIED](#)  
*Export Post-Build Ocu configurations.*
- `#define` [OCU\\_INVALID\\_CHANNEL\\_NUM](#)  
*Invalid channel number.*
- `#define` [OCU\\_INSTANCE\\_INDEX](#)  
*Instance ID of this OCU driver.*
- `#define` [OCU\\_E\\_UNINIT](#)  
*API service used without module initialization.*
- `#define` [OCU\\_E\\_PARAM\\_INVALID\\_CHANNEL](#)  
*API service used with an invalid channel Identifier.*
- `#define` [OCU\\_E\\_PARAM\\_INVALID\\_STATE](#)  
*API Ocu\_SetPinState() called with an invalid pin state or when the channel is in the RUNNING state..*
- `#define` [OCU\\_E\\_PARAM\\_INVALID\\_ACTION](#)  
*API Ocu\_SetPinAction() called with an invalid pin action.*
- `#define` [OCU\\_E\\_NO\\_VALID\\_NOTIF](#)  
*Usage of Ocu\_DisableNotification() or Ocu\_EnableNotification() on a channel where a NULL pointer is configured as the notification function.*
- `#define` [OCU\\_E\\_ALREADY\\_INITIALIZED](#)  
*API Ocu\_Init() called while the OCU driver has already been initialized.*
- `#define` [OCU\\_E\\_PARAM\\_POINTER](#)

- *API Ocu\_GetVersionInfo() is called with a NULL parameter.*
- `#define OCU_E_BUSY`  
*API Ocu\_StartChannel() called on a channel that is in state RUNNING.*
- `#define OCU_E_PARAM_NO_PIN`  
*Ocu\_SetPinState() or Ocu\_SetPinAction() called for a channel that doesnt have an associated output pin.*
- `#define OCU_E_INIT_FAILED`  
*API Ocu\_Init service called with wrong parameter.*
- `#define OCU_E_PARAM_INVALID_VALUE`  
*Ocu\_SetAbsoluteThreshold() or Ocu\_SetRelativeThreshold() called for with a compare match parameter greater than maximum supported counter value for a given channel.*
- `#define OCU_E_PARAM_CONFIG`  
*API Ocu\_Init service called with wrong Core number.*
- `#define OCU_INIT_ID`  
*API service ID of Ocu\_Init() function.*
- `#define OCU_DEINIT_ID`  
*API service ID of Ocu\_DeInit() function.*
- `#define OCU_STARTCHANNEL_ID`  
*API service ID of Ocu\_StartChannel() function.*
- `#define OCU_STOPCHANNEL_ID`  
*API service ID of Ocu\_StopChannel() function.*
- `#define OCU_SETPINSTATE_ID`  
*API service ID of Ocu\_SetPinState() function.*
- `#define OCU_SETPINACTION_ID`  
*API service ID of Ocu\_SetPinAction() function.*
- `#define OCU_GETCOUNTER_ID`  
*API service ID of Ocu\_GetCounter() function.*
- `#define OCU_SETABSOLUTETHRESHOLD_ID`  
*API service ID of Ocu\_SetAbsoluteThreshold() function.*
- `#define OCU_SETRELATIVETHRESHOLD_ID`  
*API service ID of Ocu\_SetRelativeThreshold() function.*
- `#define OCU_GETVERSIONINFO_ID`  
*API service ID of Ocu\_GetVersionInfo() function.*
- `#define OCU_DISABLENOTIFICATION_ID`  
*API service ID of Ocu\_DisableNotification() function.*
- `#define OCU_ENABLENOTIFICATION_ID`  
*API service ID of Ocu\_EnableNotification() function.*
- `#define OCU_SETCLOCKMODE_ID`  
*API Ocu\_SetClockMode service called with wrong parameter.*

## Types Reference

- `typedef void(* Ocu_NotificationType) (void)`  
*Ocu channel notification typedef.*
- `typedef uint16 Ocu_ValueType`  
*Ocu Value type (the value of the period is platform dependent and thus configurable)*
- `typedef uint16 Ocu_ChannelType`  
*Ocu channel type.*

### Enum Reference

- enum [Ocu\\_ReturnType](#)  
*Ocu Return Type.*
- enum [Ocu\\_PinActionType](#)  
*Edge Pin Action type.*
- enum [Ocu\\_PinStateType](#)  
*Pin State level.*
- enum [Ocu\\_SelectPrescalerType](#)  
*Prescaler type.*

### Function Reference

- void [Ocu\\_Init](#) (const [Ocu\\_ConfigType](#) \*ConfigPtr)  
*This function initializes the Ocu driver.*
- void [Ocu\\_StartChannel](#) ([Ocu\\_ChannelType](#) ChannelNumber)  
*This function starts a specified Ocu channel.*
- void [Ocu\\_StopChannel](#) ([Ocu\\_ChannelType](#) ChannelNumber)  
*This function stops a specified Ocu channel.*

## 6.2.2 Data Structure Documentation

### 6.2.2.1 struct [Ocu\\_ChannelConfigType](#)

Ocu high level channel configuration structure.

Definition at line 384 of file [Ocu\\_Types.h](#).

Data Fields

| Type    | Name            | Description                |
|---------|-----------------|----------------------------|
| boolean | OutputPinEnable | Channel output pin enable. |

### 6.2.2.2 struct [Ocu\\_ConfigType](#)

Ocu high level configuration structure.

Definition at line 398 of file [Ocu\\_Types.h](#).

Data Fields

- const [Ocu\\_ChannelType](#) NumChannels

- *Number of OCU channels (configured in tresos plugin builder)*  
const [Ocu\\_ChannelConfigType](#)(\* [pOcuChannelsConfig](#))[]
- *Pointer to the OCU channel configuration.*  
const [Ocu\\_Ipw\\_IpConfigType](#) [pcxIpConfig](#)
- *Combined IP specific configuration structure.*  
uint8 [CoreId](#)
- *Index Core.*

#### 6.2.2.2.1 Field Documentation

##### 6.2.2.2.1.1 NumChannels `const Ocu\_ChannelType NumChannels`

Number of OCU channels (configured in tresos plugin builder)

Definition at line 401 of file `Ocu_Types.h`.

##### 6.2.2.2.1.2 pOcuChannelsConfig `const Ocu\_ChannelConfigType(* pOcuChannelsConfig)[]`

Pointer to the OCU channel configuration.

Definition at line 403 of file `Ocu_Types.h`.

##### 6.2.2.2.1.3 pcxIpConfig `const Ocu\_Ipw\_IpConfigType pcxIpConfig`

Combined IP specific configuration structure.

Definition at line 405 of file `Ocu_Types.h`.

##### 6.2.2.2.1.4 CoreId `uint8 CoreId`

Index Core.

Definition at line 412 of file `Ocu_Types.h`.

### 6.2.3 Macro Definition Documentation

### 6.2.3.1 OCU\_STOP\_SEC\_CONFIG\_DATA\_UNSPECIFIED

```
#define OCU_STOP_SEC_CONFIG_DATA_UNSPECIFIED
```

Export Post-Build Ocu configurations.

Definition at line 165 of file Ocu.h.

### 6.2.3.2 OCU\_INVALID\_CHANNEL\_NUM

```
#define OCU_INVALID_CHANNEL_NUM
```

Invalid channel number.

Definition at line 120 of file Ocu\_Types.h.

### 6.2.3.3 OCU\_INSTANCE\_INDEX

```
#define OCU_INSTANCE_INDEX
```

Instance ID of this OCU driver.

Definition at line 123 of file Ocu\_Types.h.

### 6.2.3.4 OCU\_E\_UNINIT

```
#define OCU_E_UNINIT
```

API service used without module initialization.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 131 of file Ocu\_Types.h.

#### 6.2.3.5 OCU\_E\_PARAM\_INVALID\_CHANNEL

```
#define OCU_E_PARAM_INVALID_CHANNEL
```

API service used with an invalid channel Identifier.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 138 of file Ocu\_Types.h.

#### 6.2.3.6 OCU\_E\_PARAM\_INVALID\_STATE

```
#define OCU_E_PARAM_INVALID_STATE
```

API Ocu\_SetPinState() called with an invalid pin state or when the channel is in the RUNNING state..

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 145 of file Ocu\_Types.h.

#### 6.2.3.7 OCU\_E\_PARAM\_INVALID\_ACTION

```
#define OCU_E_PARAM_INVALID_ACTION
```

API Ocu\_SetPinAction() called with an invalid pin action.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 152 of file Ocu\_Types.h.

#### 6.2.3.8 OCU\_E\_NO\_VALID\_NOTIF

```
#define OCU_E_NO_VALID_NOTIF
```

Usage of Ocu\_DisableNotification() or Ocu\_EnableNotification() on a channel where a NULL pointer is configured as the notification function.

Errors and exceptions that will be detected by the OCU driver. AUTOSAR

Definition at line 160 of file Ocu\_Types.h.

### 6.2.3.9 OCU\_E\_ALREADY\_INITIALIZED

```
#define OCU_E_ALREADY_INITIALIZED
```

API [Ocu\\_Init\(\)](#) called while the OCU driver has already been initialized.

Errors and exceptions that will be detected by the OCU driver. AUTOSAR

Definition at line 167 of file [Ocu\\_Types.h](#).

### 6.2.3.10 OCU\_E\_PARAM\_POINTER

```
#define OCU_E_PARAM_POINTER
```

API [Ocu\\_GetVersionInfo\(\)](#) is called with a NULL parameter.

Errors and exceptions that will be detected by the OCU driver. AUTOSAR

Definition at line 174 of file [Ocu\\_Types.h](#).

### 6.2.3.11 OCU\_E\_BUSY

```
#define OCU_E_BUSY
```

API [Ocu\\_StartChannel\(\)](#) called on a channel that is in state RUNNING.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 181 of file [Ocu\\_Types.h](#).

### 6.2.3.12 OCU\_E\_PARAM\_NO\_PIN

```
#define OCU_E_PARAM_NO_PIN
```

[Ocu\\_SetPinState\(\)](#) or [Ocu\\_SetPinAction\(\)](#) called for a channel that doesn't have an associated output pin.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 189 of file [Ocu\\_Types.h](#).

### 6.2.3.13 OCU\_E\_INIT\_FAILED

```
#define OCU_E_INIT_FAILED
```

API Ocu\_Init service called with wrong parameter.

Errors and exceptions that will be detected by the OCU driver AUTOSAR

Definition at line 197 of file Ocu\_Types.h.

### 6.2.3.14 OCU\_E\_PARAM\_INVALID\_VALUE

```
#define OCU_E_PARAM_INVALID_VALUE
```

Ocu\_SetAbsoluteThreshold() or Ocu\_SetRelativeThreshold() called for with a compare match parameter greater than maximum supported counter value for a given channel.

Errors and exceptions that will be detected by the OCU driver Non-AUTOSAR

Definition at line 205 of file Ocu\_Types.h.

### 6.2.3.15 OCU\_E\_PARAM\_CONFIG

```
#define OCU_E_PARAM_CONFIG
```

API Ocu\_Init service called with wrong Core number.

Errors and exceptions that will be detected by the OCU driver when Non-AUTOSAR

Definition at line 212 of file Ocu\_Types.h.

### 6.2.3.16 OCU\_INIT\_ID

```
#define OCU_INIT_ID
```

API service ID of [Ocu\\_Init\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 218 of file Ocu\_Types.h.



### 6.2.3.17 OCU\_DEINIT\_ID

```
#define OCU_DEINIT_ID
```

API service ID of `Ocu_DeInit()` function.

Parameters used when raising an error/exception

Definition at line 224 of file `Ocu_Types.h`.

### 6.2.3.18 OCU\_STARTCHANNEL\_ID

```
#define OCU_STARTCHANNEL_ID
```

API service ID of [Ocu\\_StartChannel\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 230 of file `Ocu_Types.h`.

### 6.2.3.19 OCU\_STOPCHANNEL\_ID

```
#define OCU_STOPCHANNEL_ID
```

API service ID of [Ocu\\_StopChannel\(\)](#) function.

Parameters used when raising an error/exception

Definition at line 236 of file `Ocu_Types.h`.

### 6.2.3.20 OCU\_SETPINSTATE\_ID

```
#define OCU_SETPINSTATE_ID
```

API service ID of `Ocu_SetPinState()` function.

Parameters used when raising an error/exception

Definition at line 241 of file `Ocu_Types.h`.

#### 6.2.3.21 OCU\_SETPINACTION\_ID

```
#define OCU_SETPINACTION_ID
```

API service ID of Ocu\_SetPinAction() function.

Parameters used when raising an error/exception

Definition at line 247 of file Ocu\_Types.h.

#### 6.2.3.22 OCU\_GETCOUNTER\_ID

```
#define OCU_GETCOUNTER_ID
```

API service ID of Ocu\_GetCounter() function.

Parameters used when raising an error/exception

Definition at line 253 of file Ocu\_Types.h.

#### 6.2.3.23 OCU\_SETABSOLUTETHRESHOLD\_ID

```
#define OCU_SETABSOLUTETHRESHOLD_ID
```

API service ID of Ocu\_SetAbsoluteThreshold() function.

Parameters used when raising an error/exception

Definition at line 259 of file Ocu\_Types.h.

#### 6.2.3.24 OCU\_SETRELATIVETHRESHOLD\_ID

```
#define OCU_SETRELATIVETHRESHOLD_ID
```

API service ID of Ocu\_SetRelativeThreshold() function.

Parameters used when raising an error/exception

Definition at line 265 of file Ocu\_Types.h.

### 6.2.3.25 OCU\_GETVERSIONINFO\_ID

```
#define OCU_GETVERSIONINFO_ID
```

API service ID of Ocu\_GetVersionInfo() function.

Parameters used when raising an error/exception

Definition at line 271 of file Ocu\_Types.h.

### 6.2.3.26 OCU\_DISABLENOTIFICATION\_ID

```
#define OCU_DISABLENOTIFICATION_ID
```

API service ID of Ocu\_DisableNotification() function.

Parameters used when raising an error/exception

Definition at line 277 of file Ocu\_Types.h.

### 6.2.3.27 OCU\_ENABLENOTIFICATION\_ID

```
#define OCU_ENABLENOTIFICATION_ID
```

API service ID of Ocu\_EnableNotification() function.

Parameters used when raising an error/exception

Definition at line 283 of file Ocu\_Types.h.

### 6.2.3.28 OCU\_SETCLOCKMODE\_ID

```
#define OCU_SETCLOCKMODE_ID
```

API Ocu\_SetClockMode service called with wrong parameter.

Parameters used when raising an error/exception

Definition at line 289 of file Ocu\_Types.h.

## 6.2.4 Types Reference

#### 6.2.4.1 Ocu\_NotificationType

```
typedef void(* Ocu_NotificationType) (void)
```

Ocu channel notification typedef.

The callback notifications shall be configurable as pointers to user defined functions within the configuration structure.

Definition at line 304 of file Ocu\_Types.h.

#### 6.2.4.2 Ocu\_ValueType

```
typedef uint16 Ocu_ValueType
```

Ocu Value type (the value of the period is platform dependent and thus configurable)

Type for reading the counter and writing the threshold values (in number of ticks)

Definition at line 311 of file Ocu\_Types.h.

#### 6.2.4.3 Ocu\_ChannelType

```
typedef uint16 Ocu_ChannelType
```

Ocu channel type.

Numeric identifier of an OCU channel.

Definition at line 317 of file Ocu\_Types.h.

### 6.2.5 Enum Reference

#### 6.2.5.1 Ocu\_ReturnType

```
enum Ocu_ReturnType
```

Ocu Return Type.

Return information after setting a new threshold value.

Enumerator

|                         |                                                                         |
|-------------------------|-------------------------------------------------------------------------|
| OCU_CM_IN_REF_INTERVAL  | The compare match will occur inside the current Reference Interval.     |
| OCU_CM_OUT_REF_INTERVAL | The compare match will not occur inside the current Reference Interval. |

Definition at line 326 of file Ocu\_Types.h.

### 6.2.5.2 Ocu\_PinActionType

enum `Ocu_PinActionType`

Edge Pin Action type.

Automatic action (by hardware) to be performed on a pin attached to an OCU channel.

Enumerator

|              |                                                                                           |
|--------------|-------------------------------------------------------------------------------------------|
| OCU_SET_LOW  | The channel pin will be set LOW upon compare match.                                       |
| OCU_SET_HIGH | The channel pin will be set HIGH upon compare match.                                      |
| OCU_TOGGLE   | The channel pin will be set to the opposite of its current level HIGH upon compare match. |
| OCU_DISABLE  | The channel pin will remain at its current level upon compare match.                      |

Definition at line 339 of file Ocu\_Types.h.

### 6.2.5.3 Ocu\_PinStateType

enum `Ocu_PinStateType`

Pin State level.

Output state of the pin linked to an OCU channel.

Enumerator

|          |                              |
|----------|------------------------------|
| OCU_LOW  | Ocu Pin level is logic low.  |
| OCU_HIGH | Ocu Pin level is logic high. |

Definition at line 358 of file Ocu\_Types.h.

### 6.2.5.4 Ocu\_SelectPrescalerType

enum `Ocu_SelectPrescalerType`

Prescaler type.

This enum specifies the possible types of prescallers used to configure base-clock timers

Enumerator

|                                        |                                                         |
|----------------------------------------|---------------------------------------------------------|
| <code>OCU_PRIMARY_PRESCALER</code>     | Selected value is the default/primary prescaler.        |
| <code>OCU_ALTERNATIVE_PRESCALER</code> | Selected value is the alternative configured prescaler. |

Definition at line 371 of file `Ocu_Types.h`.

## 6.2.6 Function Reference

### 6.2.6.1 Ocu\_Init()

```
void Ocu_Init (
 const Ocu_ConfigType * ConfigPtr)
```

This function initializes the Ocu driver.

The function `Ocu_Init` shall initialize all internals variables and the used OCU structure of the microcontroller according to the parameters specified in `ConfigPtr`. Ocu shall initialize the free-running timers that are used by the driver.

If development error detection is enabled, calling the routine with a NULL `ConfigPtr`, `Ocu_Init` shall raise the development error `OCU_E_INIT_FAILED` and return without any action.

If development error detection is enabled, calling the routine `Ocu_Init` while the OCU driver and hardware are already initialized will cause a development error `OCU_E_ALREADY_INITIALIZED`. The desired functionality shall be left without any action.

For pre-compile and link time configuration variants, a NULL pointer shall be passed to the initialization routine. In this case the check for this NULL pointer has to be omitted.

If development error detection for the Ocu module is enabled, if any function (except `Ocu_Init`) is called before `Ocu_Init` has been called, the called function shall raise development error `OCU_E_UNINIT`.

Parameters

|    |                  |                                            |
|----|------------------|--------------------------------------------|
| in | <i>ConfigPtr</i> | pointer to OCU top configuration structure |
|----|------------------|--------------------------------------------|

Returns

void

### 6.2.6.2 Ocu\_StartChannel()

```
void Ocu_StartChannel (
 Ocu_ChannelType ChannelNumber)
```

This function starts a specified Ocu channel.

The function Ocu\_StartChannel shall start an OCU channel by allowing all compare match configured actions to be performed.

The state of the selected channel shall be set to RUNNING If the function Ocu\_StartChannel has been successfully performed.

If development error detection is enabled for the OCU driver: If the function Ocu\_StartChannel is called on a channel in the state RUNNING, then the function shall raise the error OCU\_E\_BUSY and return without any action.

If development error detection is enabled for the OCU driver: If the parameter ChannelNumber is invalid (not within the range specified by the configuration), the function Ocu\_StartChannel shall raise the error OCU\_E\_PARAM←\_INVALID\_CHANNEL and return without any action.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function Ocu\_←StartChannel shall raise the error OCU\_E\_UNINIT and return without any action.

Parameters

|    |               |                |
|----|---------------|----------------|
| in | ChannelNumber | Ocu channel Id |
|----|---------------|----------------|

Returns

void

Precondition

Ocu\_Init

### 6.2.6.3 Ocu\_StopChannel()

```
void Ocu_StopChannel (
 Ocu_ChannelType ChannelNumber)
```

This function stops a specified Ocu channel.

The function `Ocu_StopChannel` shall stop an OCU channel by halting compare match configured actions for this channel.

The state of the selected channel shall be set to STOPPED if the function `Ocu_StopChannel` is successfully performed. If the function `Ocu_StopChannel` is called on a channel in the state STOPPED, then the function shall leave without any action (no change of the channel state), and shall not raise a development error.

If development error detection is enabled for the OCU driver: If the parameter `ChannelNumber` is invalid (not within the range specified by the configuration), the function `Ocu_StopChannel` shall raise the error `OCU_E_PARAM_←_INVALID_CHANNEL` and return without any action.

If development error detection is enabled for the OCU driver: If the driver is not initialized, the function `Ocu_←StopChannel` shall raise the error `OCU_E_UNINIT` and return without any action.

Parameters

|    |                      |                  |
|----|----------------------|------------------|
| in | <i>ChannelNumber</i> | - Ocu channel Id |
|----|----------------------|------------------|

Returns

void

Precondition

`Ocu_Init`



**How to Reach Us:**

**Home Page:**

[nxp.com](http://nxp.com)

**Web Support:**

[nxp.com/support](http://nxp.com/support)

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: [nxp.com/SalesTermsandConditions](http://nxp.com/SalesTermsandConditions).

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

